

Project Draft Design

Scope Statement

REKOLT needs a desktop console tool to replace a manual weigh-slip-and-calculator process. It should capture each delivery, price it consistently against a fixed rule set, answer season-level questions on demand, and produce one printed statement per farmer at season's end.

Out of scope at this stage: persistence between runs, multi-user access, any currency besides MUR, any crop beyond the four named in the brief. These boundaries may be revisited once the payment rules are studied more closely.

Assumptions

- One console session = one season; restarting starts fresh.
- Only one operator uses the console at a time.
- The four crop codes and their prices don't change mid-season.
- A rejected load probably still needs to be visible in volume stats even though it earns nothing, needs confirming against the brief's exact wording once payment rules are studied in detail.

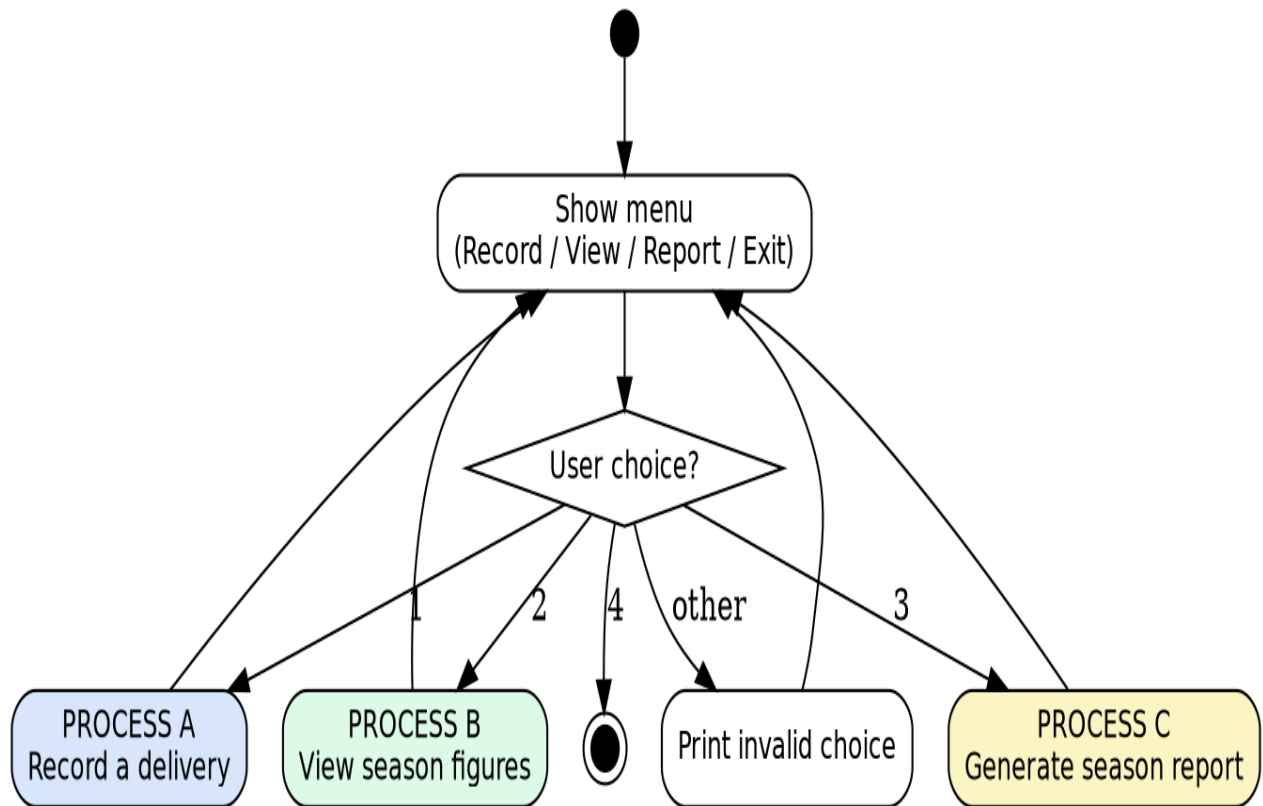
Functional Requirements

1. Capture a delivery: farmer, crop, weight, quality score, week.
2. Reject bad input without crashing; re-ask instead.
3. Work out what a delivery is worth using the rules given (grade, category, cuts).
4. A rejected-quality load still gets logged for volume, but is worth nothing.
5. Show running totals per farmer on request.
6. Show a week-by-crop volume breakdown on request.
7. Show the highest-value deliveries on request.
8. Produce one Word file with a printed section per farmer.
9. The season report's grand total must match summing every farmer's section.
10. Keep a log of when the report was generated.

Non-Functional Requirements

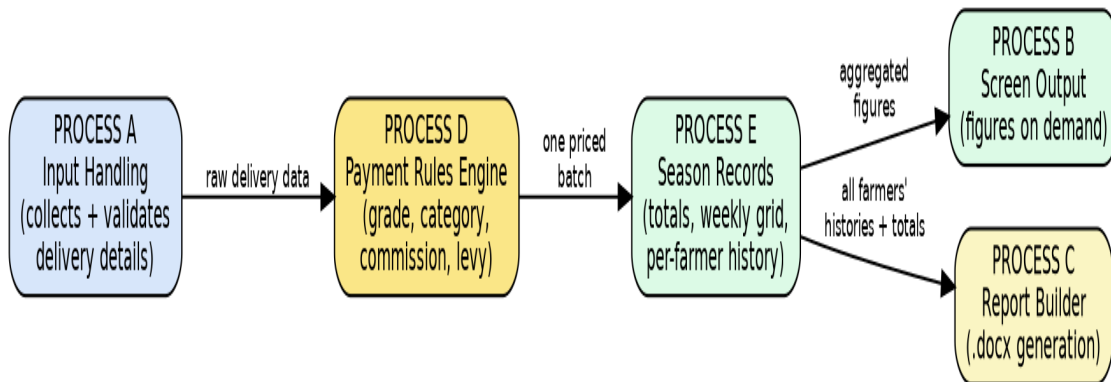
1. Never crash on bad typed input.
2. Only round money for display, never mid-calculation.
3. Keep console code, business rules, and data separated, exact package boundaries still to be finalised.
4. Processing a mixed batch of crop items should not need to check what kind each one is.

Flowchart - Overall flow



Process & Connections Map

Reading the flowchart's three lettered processes (A, B, C) plus the pricing logic they all lean on, this is a first attempt at grouping the program into bigger chunks and showing what data moves between them, before deciding whether each chunk becomes one class or several.

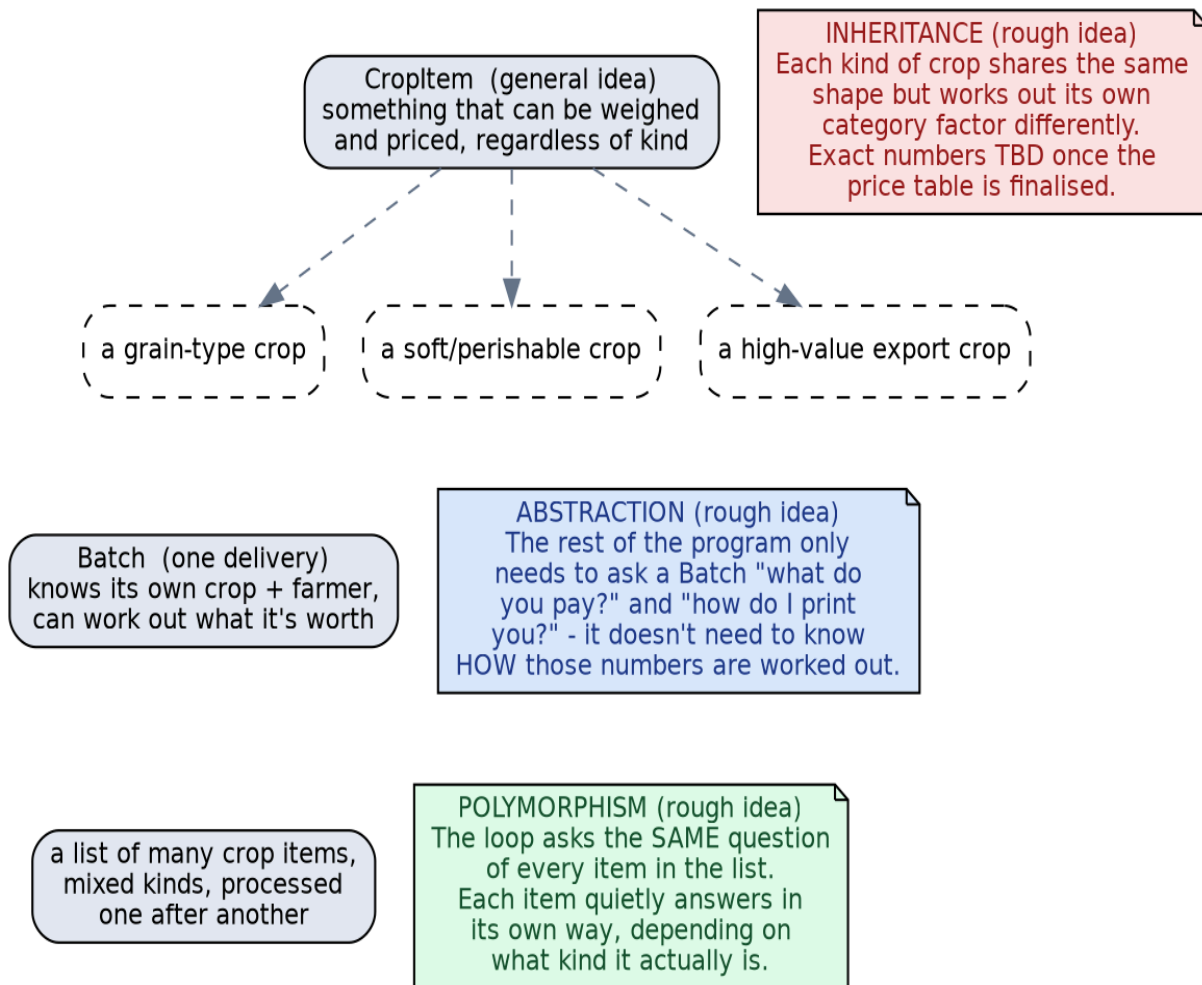


Process D (Payment Rules Engine) and Process E (Season Records) weren't visible as separate boxes on the flowchart itself, they emerged once it became clear that "record a delivery" (Process A) actually hands off to pricing logic, which then hands off to season-wide bookkeeping, before anything reaches the screen (B) or the report (C).

Noun–Verb Analysis

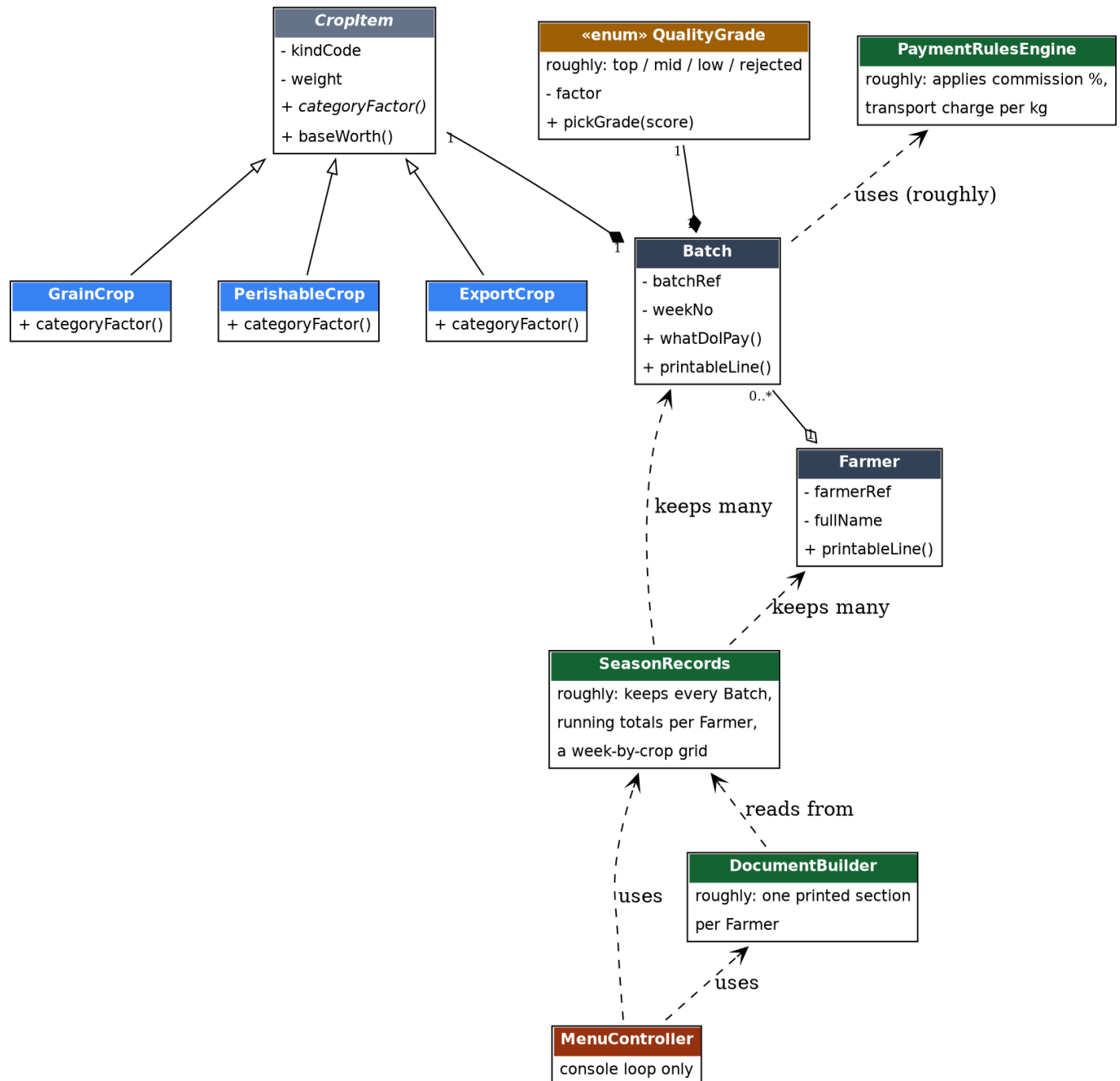
Candidate noun	Keep / reject (early reasoning)
Crop item (a delivered batch of one crop)	Keep, clearly has behaviour: knows its own worth
Farmer	Keep, identity, and the report is organised around them
Batch / delivery record	Keep, ties a farmer to a crop item for one transaction
Quality grade	Keep, though whether it's a class or just a lookup table is still open
Transaction	Leaning toward reject, looks identical to Batch, no distinct behaviour spotted yet
The cooperative itself	Leaning toward reject, only one exists, doesn't seem to need its own class
Price list	Undecided, might be a small class, might just be a constant table

Where Abstraction, Inheritance and Polymorphism Will Go.



Preliminary Class Diagram

First-pass structure only: attribute and method names here are working labels, not final. Notice this version still keeps a separate `PaymentRulesEngine` class; whether that survives contact with actual code, or gets folded into `Batch` instead, is an open question to revisit once building starts.



Pseudocode: Net Payment

```
roughly:
  find price + category factor for this crop kind
  baseWorth = weight * price
  grade = pickGrade(qualityScore)

  if grade is REJECTED:
    pay nothing, charge nothing    <- confirm this against the brief's
                                   exact wording before building,
                                   the transport charge in
particular                                   looks like it could trip this up

  else:
    value = baseWorth * grade.factor * categoryFactor
    pay = value - (5% cut) - (transport charge based on weight)
```

Data Dictionary

Field (working name)	Roughly what it is	Still to confirm
farmerRef	some identifier for a farmer	exact format not yet fixed
weight	how much was delivered	units, decimal or whole?
qualityScore	the clerk's score for the load	confirm it's always a whole number
weekNo	which week of the season	confirm season length (looks like ~20 weeks)
kindCode	which of the 4 crops	confirm the exact 4 codes

Early Rationale

Keeping a separate PaymentRulesEngine, for now

At this stage it seems cleaner to keep pricing logic away from Batch entirely, so Batch stays a simple data holder. This might turn out to be unnecessary once real code is written, if Batch ends up needing the pricing logic directly anyway, this class may get folded in.

One CropItem type, not a field with a switch

Using subtypes per crop category, rather than one class with a category field, is the current leaning, it should stop a crop being built without a valid category. This needs to hold up once the actual four crop codes and their categories are mapped out precisely.

Undecided: does Farmer need its own list of Batches, or does something else track that?

Two options are still open: Farmer keeps its own list of deliveries, or a season-wide tracker keeps everything and looks things up by farmer. No decision yet — likely to be settled once it's clearer how the season report actually needs to pull this data together.